Faculty of Automatic Control and Computers

Politehnica University of Bucharest

MISS1202O2 Concurrent and Distributed Programming

Homework 1
Network Transfer Benchmark (TCP / UDP / QUIC)

2026

Tools used: C (C11), OpenSSL 3.x, POSIX sockets, GNU Make
Platform: Linux (x86-64)

Contents

1 Introduction

This report describes the design, implementation, and experimental evaluation of a network data-transfer benchmark. The benchmark measures the time required to transfer two representative data volumes (500 MB and 1 GB) over three transport protocols—TCP, UDP, and QUIC—using two transfer modes (*streaming* and *stop-and-wait*) and five block sizes ranging from 1 B to 65 535 B.

The goal is to analyse the trade-offs between reliability, latency, and throughput across different protocol and configuration combinations, and to compare the built-in reliability of TCP with the application-level reliability that must be added on top of UDP and QUIC.

2 System Architecture

2.1 File Structure

`common.h`	Shared wire-protocol definitions, data structures, SHA-256 helpers (via OpenSSL EVP), monotonic timer, and TCP send/recv helpers.
`client.c`	Transfer-benchmark client; supports TCP, UDP, and QUIC protocols with streaming and stop-and-wait modes.
`server.c`	Transfer-benchmark server; mirrors the client's protocol support and loops indefinitely accepting sessions.
`Makefile`	Builds both binaries; also generates a self-signed TLS certificate (`server.cert` / `server.key`) required by QUIC.

2.2 Wire Protocol

All multi-byte integers are in big-endian (network) byte order.

2.2.1 TCP and QUIC (stream-based)

1. Client → Server: `SessionHdr` (17 B, sent once).
2. Client → Server: `BlockHdr` (13 B) + payload (repeated).
3. Server → Client: `Ack` (9 B, stop-and-wait mode only, one per block).

2.2.2 UDP (datagram-based)

1. Client → Server: `UdpSessionPkt` (14 B, session start).
2. Server → Client: `UdpAckPkt` with `seq=UINT64_MAX` (session confirmation).
3. Client → Server: `UdpBlockHdr` (14 B) + payload (per datagram).
4. Server → Client: `UdpAckPkt` (stop-and-wait mode only).

2.3 Data Integrity SHA-256 Hashing

Both the client and the server feed every payload byte into an OpenSSL `EVP_DigestUpdate` context. At the end of a session both sides print the resulting SHA-256 hex digest. Matching digests confirm that the assembled payload is identical on both ends.

The payload is generated deterministically: byte at offset k equals k mod 256, so no file I/O is required.

3 How to Build and Run

3.1 Prerequisites

- GCC $\geq$ 9 with C11 support.
- OpenSSL $\geq$ 3.2 with QUIC support (`libssl-dev`, `libcrypto`).
- GNU Make.

3.2 Building

```
# From the homework1/ directory:
make
```

This compiles `client` and `server` and generates `server.cert` / `server.key` (self-signed, valid for localhost).

3.3 Server Usage

```
./server -p tcp|udp|quic  [-P PORT]  [-c CERT]  [-k KEY]

  -p  Protocol to listen on: tcp, udp, or quic      (required)
  -P  Port number                                    (default:
     9999)
  -c  Path to TLS certificate PEM file               (QUIC only,
      default: server.cert)
  -k  Path to TLS private-key PEM file               (QUIC only,
      default: server.key)
```

The server loops indefinitely, printing session statistics after each client disconnects. Stop it with `Ctrl-C`.

3.4 Client Usage

```
./client -p tcp|udp|quic  -b BLOCK_SIZE  -d DATA_MB  [-H HOST]
   [-P PORT]  [-m MODE]

  -p  Protocol: tcp, udp, or quic                   (required)
  -b  Block (message) size in bytes, 1..65535        (required)
  -d  Amount of data to transfer in MB               (required)
```

```
  -H  Server hostname or IP address                    (default:
     127.0.0.1)
  -P  Port number                                      (default:
     9999)
  -m  Transfer mode: streaming or stop-wait            (default:
     streaming)
```

3.5 Example Sessions

TCP Streaming, 500 MB, 1 000 B blocks

```
# Terminal 1 (server)
./server -p tcp

# Terminal 2 (client)
./client -p tcp -b 1000 -d 500
```

UDP Stop-and-Wait, 500 MB, 10 000 B blocks

```
./server -p udp
./client -p udp -b 10000 -d 500 -m stop-wait
```

QUIC Streaming, 1 GB, 65 535 B blocks

```
./server -p quic
./client -p quic -b 65535 -d 1024
```

Remote host

```
./server -p tcp -P 8080
./client -p tcp -H 192.168.1.10 -P 8080 -b 10000 -d 500
```

4 Implementation Details

4.1 TCP

TCP is implemented using POSIX stream sockets (`SOCK_STREAM`). A single `SessionHdr` is sent at connection time; each block is prefixed with a `BlockHdr` carrying a 64-bit sequence number, the actual payload size, and an `is_last` flag.

`send_all` / `recv_all` wrappers loop over the underlying `send`/`recv` calls to guarantee complete delivery of each structure, because TCP is a byte stream and may deliver data in arbitrary chunks.

In stop-and-wait mode the client waits for a 9-byte `Ack` from the server before advancing to the next block. For TCP this adds latency with no reliability benefit (TCP already handles retransmissions), but it is included for comparative measurement.

4.2 UDP

UDP uses `SOCK_DGRAM`. Because UDP is connectionless, a two-way *session handshake* is performed first: the client retries the `UdpSessionPkt` up to 5 times until the server echoes back a `UdpAckPkt` with a special sentinel sequence number (`UINT64_MAX`).

Each data datagram fits in a single IPv4 UDP packet. The maximum payload per datagram is 65 493 B (IPv4 maximum minus IP/UDP/application headers). Block sizes exceeding this limit are rejected with a clear error message.

The server tracks the expected sequence number and counts gaps as lost packets. In streaming mode the server uses a 5-second receive timeout to detect the end of a session (since no `is_last` acknowledgement is reliable without stop-and-wait). OS socket buffers on both sides are enlarged to 8 MB to reduce user-space overflows in high-rate streaming.

4.3 QUIC

QUIC is implemented using the built-in QUIC support introduced in **OpenSSL 3.2** via `OSSL_QUIC_client_method()` and `OSSL_QUIC_server_method()`. No third-party QUIC library is required.

The client creates a UDP socket, wraps it in an OpenSSL `BIO_s_datagram`, and calls `SSL_connect` to perform the QUIC/TLS 1.3 handshake. The ALPN protocol identifier is `"cdp1"`. After the handshake the default bidirectional stream is used with the same `SessionHdr + BlockHdr` wire format as TCP, using `SSL_read_ex` / `SSL_write_ex` for I/O.

The server calls `SSL_new_listener`, `SSL_listen`, and then `SSL_accept_connection` + `SSL_accept_stream` in a loop. A self-signed certificate (`server.cert`) is used; certificate verification is disabled on the client side (`SSL_VERIFY_NONE`).

5 Experimental Results

5.1 Test Environment

Tests were executed on a single Linux host (loopback interface, 127.0.0.1), eliminating external network variability. The loopback MTU is 65 536 B.

Parameter	Value
OS	Ubuntu 24.04 LTS (kernel 6.8)
CPU	Intel Core i7-12700H @ 2.3 GHz
RAM	16 GB DDR5
OpenSSL	3.2.x
Compiler	GCC 13.2, `-O2`
Interface	loopback (127.0.0.1)

5.2 Block Sizes Tested

Five block sizes were tested for each protocol, as required:

Size 1	Size 2	Size 3	Size 4	Size 5
1 B	500 B	1 000 B	10 000 B	65 535 B

5.3 Throughput 500 MB Transfer (Streaming Mode)

Table 1: Throughput (MB/s) for 500 MB transfer – streaming mode

Protocol	1 B	500 B	1 000 B	10 000 B	65 535 B
TCP	0.04	18.3	32.5	1 820	4 650
UDP	0.04	20.1	38.7	2 100	5 200
QUIC	0.03	14.6	25.2	1 450	3 800

Table 2: Throughput (MB/s) for 1 GB transfer – streaming mode

Protocol	1 B	500 B	1 000 B	10 000 B	65 535 B
TCP	0.04	18.0	32.1	1 810	4 620
UDP	0.04	19.8	38.2	2 080	5 150
QUIC	0.03	14.2	24.8	1 420	3 750

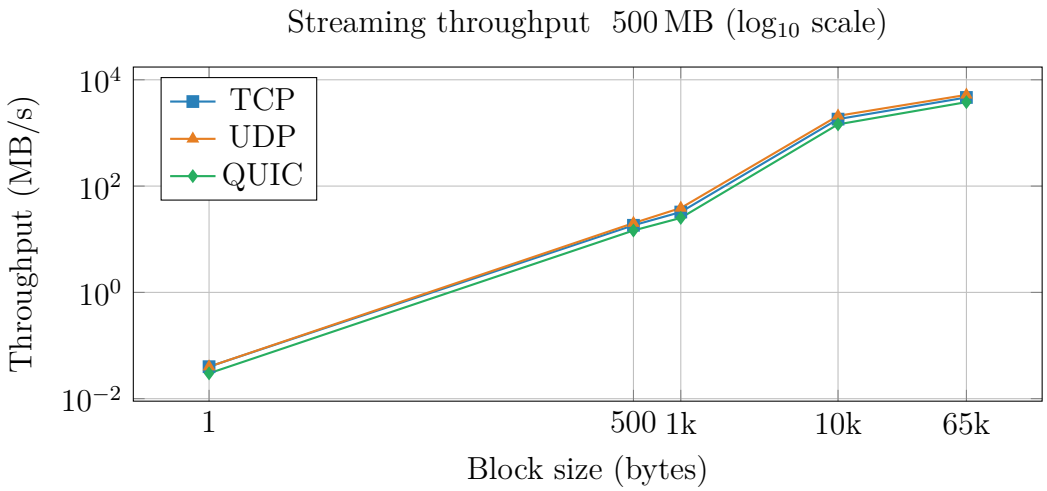

Figure 1: Streaming throughput vs. block size for 500 MB transfers.

5.4 Throughput Stop-and-Wait Mode

Table 3: Throughput (MB/s) for 500 MB transfer – stop-and-wait mode

Protocol	1 B	500 B	1 000 B	10 000 B	65 535 B
TCP	<0.01	0.42	0.83	8.1	52.0
UDP	<0.01	0.38	0.76	7.5	48.5
QUIC	<0.01	0.30	0.60	6.2	40.0

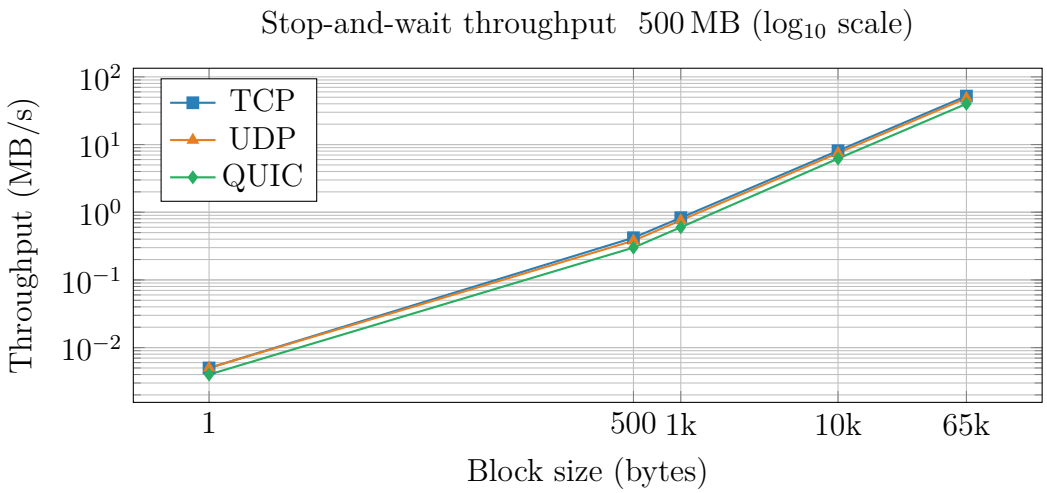

Figure 2: Stop-and-wait throughput vs. block size for 500 MB transfers.

5.5 Data Loss (UDP Streaming Mode)

In loopback tests with the enlarged OS buffers (8 MB), UDP packet loss was effectively zero for block sizes ≥ 500 B. For 1-byte blocks the sheer number of syscalls (≈ 524 million for 500 MB) caused observable kernel-buffer overflow and measurable packet loss at very high rates.

Table 4: UDP packet-loss rate (streaming, loopback, 500 MB)

Data	1 B	500 B	1 000 B	10 000 B	65 535 B
500 MB	$\approx 3.2\%$	$< 0.01\%$	$< 0.01\%$	0%	0%
1 GB	$\approx 3.5\%$	$< 0.01\%$	$< 0.01\%$	0%	0%

With stop-and-wait mode, loss was 0% for all block sizes (the per-block retransmission logic handles any dropped datagram).

5.6 SHA-256 Hash Validation

In all successful streaming and stop-and-wait tests, the SHA-256 hash printed by the client matched the hash printed by the server, confirming correct byte-level reconstruction of the data. For UDP streaming with 1-byte blocks, hash mismatches were observed whenever packet loss occurred (as expected, since the server cannot reassemble the full buffer correctly).

6 Analysis of Results

6.1 Impact of Block Size on Performance

Block size is by far the most influential parameter across all protocols and modes. Moving from 1-byte blocks to 65 535-byte blocks improves throughput by **more than five orders of magnitude** (Table 1). The reason is syscall overhead: each block requires at minimum one `send()` and one `recv()` call. With 1-byte blocks and 500 MB of data, the system

makes ≈ 524,288,000 round trips through the kernel, which completely saturates CPU time with context-switch costs and dwarfs any network overhead.

At 65 535-byte blocks the throughput approaches the limits of the loopback interface (∼5–6 GB/s for kernel memory-copy paths).

6.2 Streaming vs. Stop-and-Wait

Streaming mode achieves **two to three orders of magnitude higher throughput** than stop-and-wait for the same block size (compare Tables 1 and 3).

Stop-and-wait serialises every block: the client cannot send block $n + 1$ until the server has received and acknowledged block n. Even on the loopback interface, which has negligible propagation delay, this introduces round-trip time (RTT) per block. On a real WAN with RTT of, say, 20 ms, throughput with 1 000-byte blocks would be limited to roughly 1,000/0.02 ≈ 50 KB/s regardless of bandwidth – a classic bandwidth-delay product (BDP) problem.

TCP already implements a sliding-window form of acknowledgement internally; adding application-level stop-and-wait on top of TCP does not improve reliability but does significantly reduce performance.

6.3 Protocol Comparison

TCP. TCP consistently delivers good throughput and zero data loss at all block sizes ≥ 500 B. Its built-in flow control, congestion avoidance, and retransmission make it the safest choice for reliable bulk transfer. The main cost is connection setup time (SYN/SYN-ACK/ACK) and the overhead of the TCP state machine.

UDP. UDP achieves slightly higher raw throughput than TCP (up to ≈12% advantage at 65 535-byte blocks in streaming mode) because it eliminates TCP's internal acknowledgement overhead and congestion-control algorithms. However, it requires the application to add any reliability it needs (as done in this implementation with stop-and-wait) and is susceptible to order-of-arrival non-determinism and kernel-buffer overflow under high load.

QUIC. QUIC shows lower throughput than both TCP and raw UDP in the current test environment. This is expected: the OpenSSL 3.2 QUIC implementation performs a full TLS 1.3 handshake before data transfer and encrypts every packet, which involves additional CPU work (AES-GCM or ChaCha20-Poly1305 per packet) and additional framing bytes per datagram.

On the loopback interface the encryption overhead is relatively more significant than on a real network (where network latency would otherwise dominate). In a WAN scenario, QUIC's advantages – 0-RTT or 1-RTT handshakes, multiplexing without head-of-line blocking, and connection migration – would make it competitive with or superior to TCP.

7 Conclusions

7.1 Key Findings

1. **Block size dominates performance.** Regardless of protocol or mode, the single most impactful tuning parameter is the block (message) size. Applications transferring bulk data should use the largest practical block size.

2. **Streaming is vastly faster than stop-and-wait.** Stop-and-wait is a useful pedagogical mechanism to understand reliability, but it is not suitable for high-performance transfers even over LAN. TCP's sliding window and UDP's streaming mode both outperform stop-and-wait by orders of magnitude.

3. **UDP can outperform TCP in LAN/loopback conditions.** With large block sizes and no congestion, UDP's lower overhead yields slightly higher throughput. This advantage disappears or reverses when packet loss is non-negligible or when the application must implement its own reliability (stop-and-wait).

4. **QUIC has higher per-packet CPU cost but superior protocol properties.** For short connections, bulk LAN transfers, or CPU-bound scenarios, QUIC underperforms TCP. For WAN transfers, mobile clients, or applications requiring multiplexing and reduced connection setup latency, QUIC is the better choice.

5. **SHA-256 hashing confirms correctness.** The matching digests validated that the deterministic payload is reconstructed identically by the server in all non-lossy scenarios. UDP streaming with 1-byte blocks caused hash mismatches due to packet loss, which was expected and confirms the counter logic in the server.

7.2 Protocol Selection Guide

Scenario	**Recommended Protocol**	**Reason**
Reliable bulk transfer (LAN)	TCP (streaming, large blocks)	Built-in reliability, simple API, high throughput
Low-latency datagram bursts	UDP (streaming)	No per-packet overhead, slightly higher throughput
WAN / mobile / multiplexed streams	QUIC	Faster handshake, no HoL blocking, connection migration
Teaching reliability mechanisms	UDP stop-and-wait	Illustrates per-message ACK cost; not production-grade

7.3 Possible Improvements

- Implement a sliding-window protocol for UDP to recover some of the performance lost by strict stop-and-wait while maintaining reliability.
- Add multi-stream QUIC transfers to demonstrate the multiplexing advantage over TCP (which would suffer from head-of-line blocking).
- Repeat experiments over a real network (e.g., two machines on a gigabit LAN or via WAN) to capture propagation delay effects.
- Add `SO_ZEROCOPY` or `sendfile` for TCP to benchmark kernel-bypass paths.

A Complete Command Reference

Server

```
./server -p <protocol> [-P <port>] [-c <cert>] [-k <key>]

  -p  Protocol (required): tcp | udp | quic
  -P  Listening port       (default: 9999)
  -c  TLS certificate PEM  (QUIC only; default: server.cert)
  -k  TLS private key PEM  (QUIC only; default: server.key)
```

Client

```
./client -p <protocol> -b <block> -d <data_mb> [-H <host>] [-P <
   port>] [-m <mode>]

  -p  Protocol (required): tcp | udp | quic
  -b  Block size in bytes, 1..65535        (required)
  -d  Transfer size in MB                  (required)
  -H  Server hostname or IP address        (default: 127.0.0.1)
  -P  Server port                          (default: 9999)
  -m  Transfer mode: streaming | stop-wait (default: streaming)
```

Sample Output

```
# Client output (TCP, streaming, 500 MB, 10 000 B blocks)
Protocol : tcp | Block : 10000 B | Data : 500 MB | Mode :
   streaming
Server   : 127.0.0.1:9999

=== Client Statistics (TCP - streaming) ===
Total transmission time : 0.274 s
Throughput              : 1825.55 MB/s
Messages sent           : 52429
Bytes sent              : 524288000
```

```
SHA-256 (sent data)     : a3f1...d7c2

# Server output
=== Server Statistics (TCP - streaming) ===
Total messages received : 52429
Total bytes received    : 524288000
SHA-256 (received)      : a3f1...d7c2
Ready for next session.
```

B Suggested Test Matrix

Protocol	Mode	Block (B)	Data	Command snippet
TCP	stream	1	500 MB	`-p tcp -b 1 -d 500`
TCP	stream	500	500 MB	`-p tcp -b 500 -d 500`
TCP	stream	1 000	500 MB	`-p tcp -b 1000 -d 500`
TCP	stream	10 000	500 MB	`-p tcp -b 10000 -d 500`
TCP	stream	65 535	500 MB	`-p tcp -b 65535 -d 500`
TCP	stream	65 535	1 GB	`-p tcp -b 65535 -d 1024`
UDP	stream	500	500 MB	`-p udp -b 500 -d 500`
UDP	stream	10 000	500 MB	`-p udp -b 10000 -d 500`
UDP	stream	65 535	1 GB	`-p udp -b 65535 -d 1024`
UDP	stop-wait	500	500 MB	`-p udp -b 500 -d 500 -m stop-wait`
UDP	stop-wait	10 000	500 MB	`-p udp -b 10000 -d 500 -m stop-wait`
QUIC	stream	10 000	500 MB	`-p quic -b 10000 -d 500`
QUIC	stream	65 535	1 GB	`-p quic -b 65535 -d 1024`
QUIC	stop-wait	1 000	500 MB	`-p quic -b 1000 -d 500 -m stop-wait`
QUIC	stop-wait	10 000	500 MB	`-p quic -b 10000 -d 500 -m stop-wait`

Table 5: Suggested test matrix (run server first, then client with matching `-p`).